BÁO CÁO THỰC HÀNH ESP32: HARDWARE TIMER & DEEP SLEEP

- **Họ và tên:** Lê Văn Trường
- **MSSV:** 25119064 .
- **Nhóm** : 05 .

1. Bài tập 1: So sánh Delay, Non-blocking (millis) và Hardware Timer .

- **Hiện tượng quan sát được:**
  - ☐ Dùng `delay()`: Hàm `loop()` bị chặn hoàn toàn. Đèn LED không thể nháy nếu có lệnh delay khác đang chạy.
  - ☐ Dùng `millis()` (Non-blocking): Đèn LED nháy độc lập với các tác vụ khác, tuy nhiên vẫn bị phụ thuộc vào chu kỳ quét của hàm `loop()`. Nếu hàm `loop()` bị kẹt bởi một lệnh `delay()` lớn, `millis()` cũng sẽ bị trễ nhịp.
  - ☐ Dùng Hardware Timer (Ngắt phần cứng): Đèn LED nháy cực kỳ đều đặn và chính xác. Hàm ngắt (ISR) hoạt động hoàn toàn độc lập. Ngay cả khi hàm `loop()` bị chặn bởi lệnh `delay(5000)`, ngắt timer vẫn kích hoạt đúng thời gian định trước để điều khiển LED.
- **Kết luận**:
  - ☐ Hardware Timer là giải pháp tối ưu nhất khi cần thực thi các tác vụ đòi hỏi độ chính xác cao về thời gian mà không sợ bị ảnh hưởng bởi code trong hàm `loop()`.
  - ☐ Timer phần cứng (`esp_timer`) hoạt động hoàn toàn độc lập với hàm `loop()`. Nó giúp tạo ra nhịp cực kỳ chính xác (jitter rất thấp) ngay cả khi CPU đang bận làm việc khác hoặc bị treo bởi lệnh `delay`.

2. Bài tập 2: Chế độ Ngủ sâu (Deep Sleep) và vùng nhớ RTC .

- **Hiện tượng quan sát được:**
  - ☐ Khi ESP32 đi vào Deep Sleep, CPU dừng hoạt động và hàm `loop()` không bao giờ được chạy.
  - ☐ Sau thời gian định sẵn (5 giây), ESP32 tự động thức dậy, khởi động lại từ đầu và chạy lại hàm `setup()`.

    - [ ] Biến `demThuong` bị khởi tạo lại từ 0 (in ra là 1) sau mỗi chu kỳ ngủ - thức.
    - [ ] Biến `soLanThuc` (được gắn tag `RTC_DATA_ATTR`) vẫn giữ nguyên được giá trị và cộng dồn liên tục (1, 2, 3... 25...) qua mỗi lần thức dậy.
- **Kết luận:** Deep Sleep giúp tối ưu hóa năng lượng cực tốt cho các thiết bị chạy pin. Để không bị mất dữ liệu quan trọng khi chip ngủ, ta bắt buộc phải khai báo biến đó vào vùng nhớ RTC bằng từ khóa `RTC_DATA_ATTR`.